14.1 Decisões de mapeamento da planilha — status de entrega

Bloco 14.1 Decisões de mapeamento da planilha · Sprint 14 — Migração da planilha e go-live · Data: 2026-10-26 · Branch: `main` · Ambiente: nenhum (só leitura de código e documentos; nada conectou em banco)

Resumo

As cinco decisões do card estão tomadas, por análise, e são revisáveis. Em 2026-10-26 o Breno avisou que *"a equipe não vai tomar as decisões"* e delegou: *"utilize agentes de raciocínio para tomar a melhor decisão de negócio"*. Cada decisão foi tomada por um agente de raciocínio a partir do código, do schema e dos números do card, e depois revisada em conjunto por um revisor, que conferiu as citações de arquivo e a coerência entre elas. As correções do revisor estão marcadas como **revisão:**.

Nenhuma decisão depende de ninguém da equipe para ser aplicada. Cada uma diz o **gatilho** que a faz ser revista quando a planilha real chegar.

#	DECISÃO	EM UMA LINHA
1	Clientes	Os três nomes são um cliente só, com a razão social "EF Empreendimentos Imobiliários"; nome diferente só se junta pela tabela de equivalências
2	Contratos	**Muda o que a pergunta supunha:** a importação gera um contrato por obra com proposta aprovada (o da proposta aprovada principal), a execução vai para os itens desse contrato, e as outras aprovadas viram aditivos
3	Execução	A regra do código fica: fab = ent = máx(fabricada, instalada), med = 0 salvo coluna de medida explícita; nunca deduzida da NF
4	Links do Drive	Na importação, link em texto na observação (a); a cópia para o Storage dos 32 arquivos fica no 14.6, depois do go-live, com o `baixar-drive.mjs` que já existe
5	Data de corte	Corte no fim do dia útil anterior à virada: alvo virada sexta 13/11/2026 com `--ate 2026-11-12`; planilha congelada às 9h de 13/11

No bloco em si nenhuma linha de código mudou; as decisões foram **aplicadas depois**, e a seção "O que muda no código" está marcada como aplicada (números em `14.2-status-entrega.md` e `14.3-status-entrega.md`). O mapa de colunas (seção 6) continua **provisório** até a planilha real chegar.

1. Decisão 1 — Clientes duplicados

Decisão. "Estação Fashion", "ESTAÇÃO FASHION" e "EF EMPREENDIMENTOS IMOBILIARIOS" são o mesmo cliente. No banco ele fica com a razão social, **"EF Empreendimentos Imobiliários"**, e "Estação Fashion" continua sendo o nome da obra. Diferenças de caixa, acento, pontuação e espaço se juntam

sozinhas; nome diferente de verdade (razão social, fantasia, sigla) só se junta por equivalência explícita no mapa, e só quando as grafias aparecem na mesma obra.

Por quê.

- O próprio sistema diz o que vai no nome do cliente: o formulário orienta "Razão social ou nome completo (PF)" (`src/app/(app)/clientes/cliente-form.tsx:52`). A tabela só tem `nome` , `cnpj_cpf` e `observacao` , sem coluna de fantasia (`supabase/migrations/20260511151924_revisao_schema.sql:32-50`).
- A razão social é quem contrata, recebe a NF e paga. O CNPJ, quando for preenchido, é único por empresa (`revisao_schema.sql:52-54`) e combina com a razão social, não com o fantasia.
- "Estação Fashion" é o empreendimento; "EF" é a sigla, no padrão de SPE. O nome continua na obra e é buscável em /obras.
- O impacto é pequeno: o cliente que vale é o da aba Obras (`scripts/importacao/montar-registros.mjs:42,48`); cliente diferente na proposta só gera aviso (`montar-registros.mjs:72-73`); a gravação junta pelo nome comparado sem acento e sem caixa (`scripts/importacao/gravar.mjs:46-53`).
- Juntar só por regra explícita é o seguro: `nomeCanonicoDoCliente` (`src/lib/importacao.ts:38-46`) já faz isso. Junção aproximada pode fundir SPEs de nomes parecidos e CNPJs diferentes, o erro difícil de desfazer.

Descartadas.

- Nome fantasia como nome do cliente: contraria o formulário e não combina com o CNPJ futuro.
- Três clientes separados: a conciliação por cliente fica partida.
- Junção aproximada automática (semelhança de texto, tirar "LTDA", "S.A."): funde pessoas jurídicas sem revisão.
- Coluna `nome_fantasia` por migration: schema novo para um caso; `observacao` atende.

Risco e reversão.

- "EF Empreendimentos" ser a incorporadora-mãe e não a SPE: ainda é a contratante, a junção segue certa.
- Grafia exata errada (faltando LTDA, SPE): um UPDATE do nome em /clientes, que admin e comercial podem fazer (`supabase/migrations/20260512135125_rls_clientes.sql:31`).
- Desfazer a junção é trocar o `cliente_id` de uma obra.
- A busca de /clientes não olha `observacao` (`src/app/(app)/clientes/(lista)/page.tsx:45`): quem buscar "Fashion" em /clientes não acha; em /obras acha. Perda aceita.

Gatilho para rever.

- A planilha ou as NFs trazem CNPJs diferentes para as grafias: cada CNPJ é um cliente.
- A NF ou o comprovante traz outra razão social como tomador: vale a do documento fiscal, com a grafia exata (**copiar a razão social da NF real** antes do go-live).

- A coluna Cliente traz o nome do empreendimento em quase todas as obras: rever a regra geral.
- Casos novos: usar a lista de avisos "cliente X diferente do da obra" do `montar-registros`; juntar só grafias da mesma obra e sem CNPJ conflitante; canônico = razão social, ou, sem ela, a grafia mais frequente com caixa e acento corrigidos.

revisão: a equivalência proposta listava "EF EMPREENDIMENTOS IMOBILIARIOS" como variação de "EF Empreendimentos Imobiliários". É redundante (as duas têm a mesma `chaveDeComparacao`), mas não faz mal. A outra proposta da mesma rodada usava a razão social em caixa alta como canônico; fica a grafia com caixa e acento corretos, que é a do formulário.

2. Decisão 2 — Contratos

Decisão. A importação cria **um contrato por obra que tenha proposta aprovada.** Ele nasce da proposta aprovada principal, com os itens dela copiados, e **a execução importada vai para os itens desse contrato.** As outras propostas aprovadas da mesma obra viram **aditivos** desse contrato (`propostas.contrato_aditivo_id`). Obra sem proposta aprovada não ganha contrato. Isso responde "não" à pergunta "confirmam que nada da planilha vira contrato?".

Proposta principal: a aprovada com a data de emissão mais antiga; no empate, a de maior valor; depois, o menor número. Número do contrato: `CT-<nº da proposta>`, ou o da coluna de contrato se a planilha tiver uma.

Por quê.

- **Execução só funciona em item de contrato.** A tela recusa com "Só item de contrato vigente recebe execução" (`src/app/(app)/execucao/actions.ts:302-306`), e o "criar execuções faltantes" do 7.2 só olha `contrato_id not null` (`src/app/(app)/execucao/queries.ts:13-23`). Hoje o importador grava a execução em item de proposta (`scripts/importacao/gravar.mjs:115-123`), um estado que nenhum outro caminho do sistema gera.
- **Cadastrar o contrato depois duplica a produção.** `gerar_contrato_de_proposta` copia os itens, não move (`supabase/migrations/20260923162000_gerar_contrato_de_proposta.sql:20-27` e `:110-124`). Os itens do contrato nasceriam sem execução, o "criar execuções faltantes" criaria execuções zeradas, e as 287 linhas reais ficariam presas nos itens da proposta: a mesma esquadria duas vezes na tela.
- **A regra "uma obra, um contrato vigente"** (`docs/tecnicos/regra-um-contrato-por-obra.md`) pede um por obra, não um por proposta. Com 24 propostas para 8 obras, um por proposta já entraria violando a regra, e o trigger planejado travaria os status desses contratos.
- **O aditivo já tem lugar no banco:** `propostas.contrato_aditivo_id`, com FK composta (`supabase/migrations/20261012110000_propostas_aditivo_fk.sql`).
- **Juntar os itens de várias propostas num contrato não cabe:** o índice `idx_itens_numero_contrato` recusa dois "item 1" no mesmo contrato (`revisao_schema.sql:302-304`).
- **NF, acordo e pagamento não precisam de contrato:** o vínculo é contrato ou proposta, opcional (`supabase/migrations/20260424121550_initial.sql:513-545` e `:648-677`). As NFs ficam ligadas à proposta como hoje, e os totais não mudam.

Descartadas.

- **Nenhum contrato na importação** (a premissa da pergunta, e a recomendação de uma das análises): a produção não continua pelo sistema nas obras ativas, e gerar o contrato pela tela depois divide ou duplica a execução, como acima.
- **Um contrato por proposta aprovada:** viola a regra de um por obra.
- **Um contrato por obra com os itens de todas as aprovadas:** bate no índice único e perde o rastro proposta/item.
- **Chamar a RPC `gerar_contrato_de_proposta`:** grava `created_by = auth.uid()`, nulo com a chave de serviço, e fica fora do padrão idempotente e reversível do `gravar.mjs`. A mesma lógica é reproduzida no `gravar.mjs`.

Risco e reversão.

- **"Contrato" no sistema sugere documento assinado que não existe.** Mitigação: número rastreável `CT-<proposta>`; `data_assinatura` e `doc_url` nulos (as duas colunas aceitam nulo); observação com `[importacao:<chave>]` e o texto "gerado da proposta X aprovada na planilha; sem contrato assinado registrado"; `pct_*` em 0 (o default). A equipe completa **editando** o contrato importado, e não criando outro.
- **Proposta principal errada:** o `--desfazer` apaga execução, itens de contrato e contratos antes de propostas e obras. Em produção, o contrato pode ser editado ou rescindido; a proposta fica intacta.

Gatilho para rever.

- A aba de propostas tem coluna de número de contrato: usar esse número.
- Uma obra tem duas aprovadas que são escopos independentes (cliente ou torre com NF própria): dividir a obra, não criar dois contratos.
- Nenhuma proposta aprovada, ou status vazio: aquelas obras ficam sem contrato e entram na lista de pendências manuais.
- A obra já tem contrato vigente no sistema na hora da importação: não criar outro, ligar as aprovadas a ele como aditivo.
- Produção relevante em item de aditivo: rever como item de aditivo recebe execução (limite que já existe no sistema, não é da importação).

revisão:

1. As análises divergiram: uma decidiu "nada vira contrato, gerar pela tela depois". Fica a do contrato por obra na importação, porque a outra deixa a execução dividida entre item de proposta e item de contrato justamente nas obras que continuam em produção, e o próprio 14.2 já apontava esse efeito (`14.2-status-entrega.md`, seção 6).
2. O status do contrato importado dependia de "a obra entrar concluída", mas a importação **não lê o status da obra**: o mapa tem a coluna (`mapa-planilha.json:17`), e o `montar-registros.mjs:49-52` e o `gravar.mjs:61-64` a ignoram, então toda obra entra `ativa`. Entra no checklist: mapear o status

da obra (`ativa` , `concluida` , `suspensa` , `cancelada` , `initial.sql:143`) e derivar dele o do contrato (`concluido` para obra concluída, senão `ativo`).

3. Achado lateral, **fora do 14.1**: o `progresso_itens_pct` da view `obras_com_valores` divide a medição pela soma das quantidades de **todos** os itens da obra, de proposta e de contrato (`supabase/migrations/20260512151918_progresso_itens_obra.sql:30-46`). Como o contrato copia os itens, toda obra com contrato gerado tem o denominador em dobro e o progresso para em 50%. Já vale hoje para os contratos da tela; com a decisão 2 passa a valer para as obras importadas. Registrado como pendência para a sprint, não como parte desta decisão.

3. Decisão 3 — Execução (cascata), a crítica

Decisão. Fica a regra que o código já usa: fabricado = entregue = o maior entre a fabricada e a instalada; a medição entra em **0**, salvo quando a planilha trouxer quantidade medida explícita por item. Não se deduz medição das notas fiscais (nem med = inst onde há NF, nem proporcional ao faturado). A linha que não cabe na cascata vai para as ignoradas com o motivo. A conciliação passa a avisar as obras com NF de medição e nada medido.

Por quê.

- **Medição é fato acertado com o cliente, não conta.** A coluna é "validação final com o cliente" (`initial.sql:458-461`) e alimenta o relatório que vai para o cliente aprovar (`src/app/api/relatorio/medicao/[obraId]/route.ts:5-13`).
- **Não há como ratear NF por item:** `notas_fiscais` é por obra, sem FK para item, e o `tipo` admite sinal, entrega de material, medição, instalação, faturamento direto e outro (`initial.sql:521`). Faturar sinal e material antes de medir é o normal em esquadrias.
- **Med > 0 na importação estraga o relatório de medição:** o trigger grava o histórico com `data = current_date` (`supabase/migrations/20260925120000_execucao_medicoes.sql:29` e `:47-67`), e o relatório soma as mudanças com data no período (`src/lib/medicao.ts:4-7`). Tudo apareceria "medido" no mês da importação.
- **Med = 0 é o lado reversível:** subir a medição pela tela registra quem e quando (`supabase/migrations/20260924130000_execucao_data_fim_total.sql:76-90`); baixar uma medição errada deixa histórico negativo num documento de cliente.
- **fab = ent = inst cumpre as constraints** (`initial.sql:412,429,445,461`) sem inventar nada além do inevitável: não se instala o que não foi fabricado e entregue. Cada valor deduzido vira aviso (`src/lib/importacao.ts:178-184` , `montar-registros.mjs:129`).

Descartadas.

- med = inst onde houver NF: NF é por obra e pode ser de sinal ou material.
- med proporcional ao faturado: rateio sem vínculo NF × item é ficção contábil.
- med = inst em tudo: afirma 100% validado pelo cliente, sem evidência, e esconde o que falta faturar.
- fab = ent = inst exato, ignorando a fabricada: descarta dado quando a fabricada vier maior.

- Não importar a execução: perde as 287 linhas.
- Esperar a equipe informar: descartado pelo Breno em 2026-10-26.

Advogado do diabo. Com med = 0, as obras aparecem com progresso 0% e as 287 execuções ficam na fila de medição, com R$ 627.959,83 faturados. É transitório e verdadeiro ("o sistema ainda não tem a medição"); um número falso no relatório do cliente custa mais.

Risco e reversão.

- Painel com progresso zerado até a medição apontar. Comunicar na virada.
- **Medir e faturar de novo** o que já foi medido fora do sistema: a conciliação lista as obras com NF de medição e med = 0; o financeiro e a medição registram as medições já feitas na primeira semana, antes de emitir qualquer relatório.
- Para med > 0: basta a coluna "Qtd Medida" na planilha; o mapa já a lê (`mapa-planilha.json:63`) e a cascata a respeita (`importacao.ts:170-176`). Em dev, `--desfazer` e reimportar.

Gatilho para rever.

- A planilha traz quantidade ou % medido por item, ou boletim de medição: usar o valor, com data retroativa (data de corte, ver checklist).
- NF de medição com referência a boletim ou item: derivar do boletim, nunca da NF agregada.
- A soma das NFs de medição de uma obra passa de Σ(inst × valor_unit): incoerência real, vai para a conciliação 14.3 como divergência, e a obra não é importada às cegas.

revisão: as citações de linha foram conferidas e acertadas (`importacao.ts:178-184` para os avisos, `route.ts:5-13`). Com a decisão 2, a execução passa a morar em item de contrato; a regra da cascata não muda.

4. Decisão 4 — Os 32 links do Google Drive

Decisão. Na importação, opção **(a)**: o link vai em texto na observação, como o código já faz ("Documento:" na proposta, "Arquivo:" no pagamento e no FD). A cópia para o Storage, opção (b), fica no **14.6, depois do go-live, para os 32 arquivos**, começando pelos 7 comprovantes de pagamento. O link continua na observação como origem, e o arquivo original não sai do Drive.

Por quê.

- **Já está no padrão:** `observacaoComMarca` (`src/lib/importacao.ts:214-218`), usado no pagamento e no FD (`montar-registros.mjs:189` e `:225`); proposta em `montar-registros.mjs:82` . A leitura pega o hiperlink da célula e também URL em texto puro (`scripts/importacao/ler-planilha.mjs:17-21`).
- **(b) depende de acesso que não temos:** leitura nos 32 arquivos. Amarrar a importação a isso trava o go-live por anexos que não mudam nenhum total.
- **(a) agora e (b) depois não gera retrabalho:** o `scripts/importacao/baixar-drive.mjs` (14.6) já existe, lê a planilha com os mesmos módulos do importador, acha pagamento e FD pela marca

`[importacao:…]` e proposta pelo número, e tem simulação, `--baixar-para` (prova o acesso), `--gravar` e `--desfazer`.

- **Há destino para os três tipos, sem schema novo:** `propostas.anexos` (`initial.sql:233`), `pagamentos.anexo` (`initial.sql:597`, renomeada de `comprovante_url` em `revisao_schema.sql:422`) e `fd.evidencias` (`initial.sql:819`), nos buckets privados `anexos` e `evidencias` (`supabase/migrations/20260424121552_storage_buckets.sql:15-33`).
- **Os comprovantes têm o maior valor de auditoria:** são a prova dos R$ 489.498,13 recebidos. Por isso vão primeiro.
- Segundo a documentação do Supabase, o backup do banco não inclui os objetos do Storage. Mais uma razão para o original continuar no Drive depois da cópia.

Descartadas.

- (b) agora, para os 32: bloqueia a importação por falta de acesso.
- (a) para sempre: o comprovante não pode depender da conta de alguém no Drive.
- Gravar a URL do Drive direto em `pagamentos.anexo`: a coluna é caminho no bucket, e a tela abriria e trocaria o comprovante errado.

Risco e reversão. Um link quebrar antes do 14.6: nenhum valor muda (o status da NF vem dos pagamentos), perde-se a prova. Mitigação no go-live: pedir que os arquivos passem para um Drive compartilhado da empresa; o relatório da importação lista os links. O `baixar-drive.mjs --desfazer` tira do Storage e devolve o link à observação; o `--desfazer` da importação segue funcionando.

Gatilho para rever.

- Links de conta pessoal (fora do domínio da empresa), quebrados ou sem acesso: os 7 comprovantes sobem antes do go-live, com o mesmo script.
- Link de pasta em vez de arquivo: escolher o arquivo antes de baixar.
- Número de links bem maior que 32.

revisão: a análise original dizia que o FD não tinha onde guardar o arquivo (e por isso deixava as 3 notas de FD como link de vez), que o comprovante ia em `comprovante_url`, e que a leitura só pegava hiperlink. Os três pontos estavam errados: `fd.evidencias` existe, a coluna do comprovante é `pagamentos.anexo`, e `linkDaCelula` também lê URL em texto. Com isso, e com o `baixar-drive.mjs` pronto, o 14.6 cobre os 32 arquivos, e não só 29.

5. Decisão 5 — Data de corte

Decisão. A importação leva tudo que tem data **até o fim do último dia útil antes da virada**, e dali em diante tudo se lança só no sistema. Alvo: virar na **sexta 13/11/2026** à tarde com `--ate 2026-11-12`. A planilha congela às 9h de 13/11; o que acontecer em 13/11 se lança à mão no sistema depois da virada.

A regra, para qualquer data de virada.

1. **Corte** = fim do dia útil anterior à virada, inclusive (o script só descarta o que tem data **posterior** ao corte, `montar-registros.mjs:32`).
2. **Congelamento** às 9h do dia da virada: a planilha vira somente leitura e se salva uma cópia datada (`Gestão Contratos - congelada 2026-11-13.xlsx`). A importação de produção roda sobre a cópia, nunca sobre o arquivo vivo: a chave de idempotência de pagamento e FD muda se a linha for editada (`src/lib/importacao.ts:199-208`).
3. **Entre o corte e a virada:** nada se lança na planilha. O que acontecer vai para uma lista à parte e entra pelas telas até segunda 16/11 ao meio-dia. A rede de segurança é o log de ignoradas por data (`montar-registros.mjs:152, 184, 211`).
4. **Condição para virar**, até quarta 11/11: ensaio em gc-dev com a planilha real, sem linha ignorada sem explicação; decisões 1 a 4 aplicadas. Se faltar qualquer uma, a virada passa para a próxima sexta útil. Sexta 20/11/2026 é feriado nacional (Consciência Negra), então a próxima é **sexta 27/11, com corte em 26/11**.

Por quê.

- O card do 14.1 vence em 12/11/2026, e a sexta seguinte é 13/11. A virada na sexta deixa o fim de semana para conferir.
- Corte num dia já fechado: nenhum lançamento "meio na planilha, meio no sistema".
- Obras, propostas, itens e execução não têm filtro de data e entram como estão no arquivo congelado (a execução é saldo acumulado); por isso o congelamento importa tanto quanto a data.
- NF antes do corte com pagamento depois fica coerente: nasce emitida e o trigger recalcula quando o pagamento for lançado.
- A ponte de um dia útil, lançada pelas telas, evita uma segunda importação incremental.

Descartadas.

- Corte no próprio dia da virada: o dia ainda está aberto; um lançamento da tarde se perde ou entra duas vezes.
- Sem corte, sobre o arquivo vivo: não reproduzível.
- Corte no fim do mês: perde dias de lançamento e coincide com o pico de NF.
- Período paralelo, lançando nos dois: duas fontes da verdade.
- Segunda importação incremental depois do go-live: linha corrigida na planilha vira registro novo.
- Virada no meio da semana: menos folga para refazer.

Risco e reversão.

- **FD sem data de lançamento:** o script usa o vencimento (`montar-registros.mjs:208`), e um FD lançado antes do corte com vencimento depois sai como ignorado. Mitigação no checklist.
- Alguém escrever na planilha depois do congelamento: o log de ignoradas acusa; cada linha vira lançamento manual.
- Importação de produção dar errado: `--desfazer` e rodar de novo na janela; se a conferência não fechar até domingo 18h, a planilha descongela, o sistema não é anunciado, e a virada vai para

27/11.

Os totais do card são uma foto de data desconhecida. R$ 627.959,83 (NF emitida), R$ 489.498,13 (recebido) e R$ 2.060.290,51 (FD a descontar) não são o alvo. O alvo é o que a planilha congelada dá até o corte, e o script imprime. Os do card servem de sanidade: NF emitida e recebido devem dar **maior ou igual**; se derem menor, há NF cancelada ou linha perdida no mapa, a explicar antes de virar. O FD a descontar cai quando os descontos se aplicam, então só se compara com a planilha congelada. As contagens (9 NF, 7 pagamentos, 248 FD) são pisos.

Gatilho para rever.

- Lançamentos com data futura em quantidade (NF pré-emitida, FD agendado).
- Muitas linhas de FD sem data de lançamento: a mudança do FD deixa de ser opcional.
- Movimento pesado entre corte e virada (mais de ~20 lançamentos por dia): corte no mesmo dia, mas às 18h.
- A planilha registra execução com data: o corte passa a filtrar a execução.
- Ensaio sem fechar até 11/11: virada em 27/11, corte em 26/11.

revisão: uma das análises pôs o corte na própria sexta da virada, com congelamento às 12h. Fica o corte no dia útil anterior: com o corte no mesmo dia, o script levaria o dia ainda aberto, e o que fosse lançado entre o meio-dia e a importação ficaria sem lugar definido.

6. Mapa final coluna → tabela.campo (provisório)

Provisório até a planilha real chegar. Os cabeçalhos abaixo são **suposições** feitas a partir da descrição do card; o script aceita qualquer um da lista, comparado sem acento e sem caixa, e acha a linha de cabeçalho sozinho. Fonte: `scripts/importacao/mapa-planilha.json`. Quando a planilha chegar, esta tabela é refeita com o cabeçalho real de cada coluna. (*) = obrigatório: sem a coluna, a aba não é lida.

Obras — aba "Obras" ou "Cadastro de Obras"

CABEÇALHOS ACEITOS	DESTINO	REGRA
Código, Codigo da Obra, Cód. Obra, Obra ID	`obras.codigo_obra`	vazio → `LEGADO-<nome>`, com aviso
Obra, Nome da Obra, Nome (*)	`obras.nome`	
Cliente, Construtora, Contratante (*)	`clientes.nome` → `obras.cliente_id`	canônico pela decisão 1
Cidade	`obras.cidade`	
Endereço, Local	`obras.endereco`	

CABEÇALHOS ACEITOS	DESTINO	REGRA
Status, Situação	`obras.status`	**hoje ignorado**; decisão 2 pede mapear (checklist)
Início, Data de Início	`obras.data_inicio`	

Propostas — aba "Propostas" ou "Orçamentos e Propostas"

CABEÇALHOS ACEITOS	DESTINO	REGRA
Proposta, Nº Proposta, Número, N° (*)	`propostas.numero`	chave de idempotência
Obra, Código da Obra (*)	`propostas.obra_id`	pela obra, por código ou nome
Cliente	—	só conferência: diferente do da obra vira aviso
Descrição, Escopo	`propostas.descricao`	
Data, Data de Emissão, Emissão	`propostas.data_emissao`, `data_envio`, `data_decisao`	
Valor, Valor Total, Total (*)	`propostas.valor_total`	o trigger do 5.6 faz valer a soma dos itens
Desconto	`propostas.desconto`	
Status, Situação	`propostas.status`	não reconhecido → enviada, com aviso
Condições, Condições de Pagamento, Forma de Pagamento	`propostas.condicoes_pagamento`	
Documento, Link, Arquivo, Drive	`propostas.observacao` ("Documento: ")	decisão 4
Contrato, Nº Contrato (**novo**, opcional)	`contratos.numero`	decisão 2; sem a coluna, `CT-<proposta>`

Contratos — sem aba (decisão 2)

ORIGEM	DESTINO
proposta aprovada principal da obra	`contratos` (número, obra, `proposta_origem_id`, valor = soma dos itens, desconto e condições da proposta, status pela obra)
itens da proposta principal	`itens` com `contrato_id` (cópia)
demais aprovadas da obra	`propostas.contrato_aditivo_id`

Itens — aba "Itens", "Itens da Proposta" ou "Esquadrias"

CABEÇALHOS ACEITOS	DESTINO	REGRA
Proposta, Nº Proposta (*)	`itens.proposta_id`	
Item, Nº Item, Número (*)	`itens.numero`	inteiro
Tipo	`itens.tipo`	
Descrição	`itens.descricao`	
Linha	`itens.linha`	
Acabamento, Cor	`itens.acabamento`	
Largura, L	`itens.largura`	
Altura, H	`itens.altura`	
Quantidade, Qtd, Qtde (*)	`itens.quantidade`	
Unidade, Un	`itens.unidade`	QTD ou M2; outra → ignorada
Valor Unitário, Valor Unit, Preço Unitário (*)	`itens.valor_unit`	
Localização, Local, Pavimento	`itens.localizacao`	
Vidro, Vidros	`itens.vidros`	

Execução — aba "Controle de Execução" ou "Execução"

CABEÇALHOS ACEITOS	DESTINO	REGRA
Proposta, Nº Proposta (*) + Item, Nº Item (*)	`execucao.item_id`	item do **contrato** da obra (decisão 2); aditivo ou não aprovada → item da proposta, com aviso
Quantidade, Qtd, Qtd Total	`execucao.quantidade_total`	uma linha só → a do item
Qtd Fabricada, Fabricada	`execucao.fab_qtd`	decisão 3: máx(fabricada, entregue, instalada)
Qtd Entregue, Entregue	`execucao.ent_qtd`	decisão 3: máx(entregue, instalada)
Qtd Instalada, Instalada	`execucao.inst_qtd`	
Qtd Medida, Medida	`execucao.med_qtd`	decisão 3: 0 se não vier
Localização, Local, Pavimento	`execucao.localizacao`	
Responsável, Equipe	`execucao.inst_responsavel`	

CABEÇALHOS ACEITOS	DESTINO	REGRA
Observação, Obs	`execucao.inst_observacao`	
(ordem das linhas do item)	`execucao.sequencial`	

Notas fiscais — aba "Notas Fiscais", "NF" ou "NFs"

CABEÇALHOS ACEITOS	DESTINO	REGRA
NF, Nº NF, Número, Nota (*)	`notas_fiscais.numero`	chave = número + série
Série	`notas_fiscais.serie`	
Obra (*)	`notas_fiscais.obra_id`	
Proposta	`notas_fiscais.proposta_id`	não importada → sem vínculo, com aviso
Tipo, Referente	`notas_fiscais.tipo`	tabela `tipos_nf` do mapa; o resto → outro
Emissão, Data de Emissão, Data (*)	`notas_fiscais.data_emissao`	depois do corte → ignorada
Vencimento	`notas_fiscais.data_vencimento`	
Valor, Valor Total (*)	`notas_fiscais.valor_total`	
Status, Situação	— (só cancelada é gravada)	o resto vem do trigger; o texto vai para a conciliação
Chave, Chave NF-e	`notas_fiscais.chave_nfe`	

Pagamentos — aba "Pagamentos", "Extrato" ou "Recebimentos"

CABEÇALHOS ACEITOS	DESTINO	REGRA
NF, Nº NF, Nota (*) + Série	`pagamentos.nota_id` (e `obra_id` da NF)	NF ambígua sem série → ignorada
Data, Data do Pagamento, Recebido em (*)	`pagamentos.data_pagamento`	depois do corte → ignorado
Valor, Valor Recebido (*)	`pagamentos.valor`	
Forma, Forma de Pagamento, Meio	`pagamentos.forma`	não reconhecida → outro, texto na observação

CABEÇALHOS ACEITOS	DESTINO	REGRA
Observação, Obs, Histórico	`pagamentos.observacao`	+ marca `[importacao:…]`
Comprovante, Link, Drive	`pagamentos.observacao` ("Arquivo: ")	decisão 4; no 14.6 → `pagamentos.anexo`

FD — aba "FD", "Faturamento Direto" ou "Lançamentos FD"

CABEÇALHOS ACEITOS	DESTINO	REGRA
Obra (*)	`fd.obra_id`	
Fornecedor (*)	`fd.fornecedor`	texto
Data, Data de Lançamento, Lançamento	`fd.data_lancamento`	vazio → vencimento ou pagamento, com aviso
Código	`fd.codigo`	
Especificação, Descrição, Material	`fd.especificacao`	
Quantidade, Qtd	`fd.quantidade`	
Unidade, Un	`fd.unidade`	
Preço Unitário, Valor Unitário	`fd.preco_unitario`	
Valor, Valor Total (*)	`fd.valor`	
A Descontar, Valor a Descontar, Descontar	`fd.valor_descontar`	vazio → valor inteiro, com aviso
Vencimento	`fd.data_vencimento`	
Pagamento, Data de Pagamento, Pago em	`fd.data_pagamento`	
Pedido, Nota, Documento	`fd.pedido_documento`	
Item/Parcela, Parcela	`fd.item_parcela`	
Observação, Obs	`fd.observacao`	+ marca `[importacao:…]`
Link, Drive, Nota (link)	`fd.observacao` ("Arquivo: ")	decisão 4; no 14.6 → `fd.evidencias`

A aba Resumo não é importada: o `conciliar.mjs` (14.3) a usa para comparar com as views por obra.

7. Recomendações de go-live (14.4)

São recomendações, não execução. **Decisão do Breno em 26/10: nada em gc-prod** (ele não tem acesso). O go-live fecha como ensaio em gc-dev (`14.4` e `14.5`); reativar, importar, configurar o Auth, o domínio e o monitoramento em prod são **registro para quem for operar prod**, não tarefa nossa.

(a) Monitoramento de erro. Sentry no plano gratuito, com aviso por e-mail, para as exceções do servidor e do navegador. A `auditoria_eventos` continua como trilha de negócio (a escrita recusada já vira evento `resultado = 'erro'` , `src/lib/supabase/server.ts:36-73`); o log da Vercel fica de apoio. O descoberto hoje é o erro de render e o de JavaScript no navegador. Descartados: só o log da Vercel (sem aviso, retenção curta), só a auditoria (não vê o cliente). Risco: dado pessoal em terceiro; `sendDefaultPii: false` e corpo de requisição limpo. Rever se passar da cota ou se a empresa vetar.

(b) Backup. Plano **Pro** do Supabase com o backup diário que vem nele, mais um `pg_dump` logo antes e logo depois da importação de produção, guardado fora do Supabase. Sem PITR por enquanto. O plano gratuito pausa por inatividade (o gc-dev ficou `INACTIVE` em 2026-09-21 e de novo em 2026-10-26), o que em produção é inaceitável. Os arquivos do Storage ficam fora do backup do banco; os originais legados continuam no Drive (decisão 4). Ligar o PITR se passar de ~50 lançamentos por dia ou se a automação gravar em volume. Custo do plano: decisão do Breno.

(c) Janela de virada. Sexta **13/11/2026**, conforme a decisão 5:

- **quarta 11/11:** prazo das três condições (ensaio com a planilha real em gc-dev, decisões aplicadas);
- **quinta 12/11:** último dia lançado na planilha (é o corte);
- **sexta 13/11, 9h:** planilha congelada e copiada;
- **sexta, 13h às 18h:** `pg_dump` , importação em produção, conciliação, `pg_dump` ;
- **fim de semana:** folga para refazer com `--desfazer` ; se não fechar até **domingo 18h**, aborta e a virada vai para 27/11;
- **segunda 16/11:** primeiro uso, com alguém de plantão de manhã; lançamentos de 13/11 até o meio-dia.

revisão: a recomendação original pedia que a conciliação "batesse nos três totais do card" e abortava na sexta às 18h. Pela decisão 5, os totais do card são só sanidade (≥ para NF e recebido), e o prazo de abortar é o domingo, que é para isso que a folga do fim de semana existe.

(d) Usuários e perfis. Exatamente **2 admins ativos** (responsável pela empresa e administrativo-financeiro), um perfil por função para o resto (comercial, financeiro, medição, produção; diretoria e consulta como visualizador), **uma conta por pessoa**, e usuários novos pelo pedido de acesso (`supabase/migrations/20261019100000_pedidos_acesso.sql`). O trigger só garante 1 admin ativo (`supabase/migrations/20261005120000_usuarios_protecao_perfil.sql:84-94`); com um só, a falta dele deixa a empresa sem quem crie usuário. Conta compartilhada anula a trilha da auditoria. Se a empresa tiver só uma pessoa de gestão, o segundo admin é o Breno.

(e) Domínio. Subdomínio do domínio da empresa (`<sistema>.<domínio>`) por CNAME para a Vercel, com o `*.vercel.app` de reserva. Pré-requisito: Site URL e URLs de redirecionamento do Auth de produção com o domínio novo, senão convite, definir senha e "Esqueci minha senha" apontam para o

endereço errado. Testar o convite logo depois da troca. Nada muda no repositório; muda a configuração do Auth em gc-prod e a variável de URL na Vercel, as duas por conta de quem operar prod (registro). Rever se a empresa não tiver acesso ao DNS.

8. O que muda no código

Checklist das decisões. ✅ **Aplicado em 2026-10-26** (importador: ensaio 17/17; conciliação 41/41 no ensaio; números em `14.2-status-entrega.md` e `14.3-status-entrega.md`). Exceção, **não feita**: os dois itens opcionais da decisão 1 (levar ao cliente as grafias originais, `clientes.observacao` = "Na planilha também como: ..."). Conferido no código: `grep` por "apelidos" e "Na planilha também como" em `scripts/importacao` não acha nada; a equivalência de cliente já funciona pelo mapa (`clientes.equivalencias`), só não guarda as grafias originais.

Mapa (`scripts/importacao/mapa-planilha.json`)

- [x] ✅ `clientes.equivalencias`: `{ "EF Empreendimentos Imobiliários": ["Estação Fashion"] }`, com a razão social copiada da NF real (decisão 1).
- [x] ✅ `data_corte`: `null` → `"2026-11-12"` (ou `"2026-11-26"` se a virada for para 27/11); o `--ate` continua sobrepondo nos ensaios (`importar-planilha.mjs:49`) (decisão 5).
- [x] ✅ bloco novo `"contratos": { "gerar": "um_por_obra", "prefixo_numero": "CT-", "principal": ["data_emissao_asc", "valor_desc", "numero_asc"] }` (decisão 2).
- [x] ✅ `abas.propostas.campos.contrato`: `["Contrato", "Nº Contrato"]`, opcional (decisão 2).
- [ ] (aguarda a planilha real) ao chegar a planilha: cabeçalhos reais em todas as abas, e refazer a tabela da seção 6.

Regras puras (`src/lib/importacao.ts`, com testes em `importacao.test.ts`)

- [x] ✅ `contratosDaImportacao(propostas)` → por obra `{ obra, principal, aditivos[] }`, só aprovadas, com a ordem de principal da decisão 2.
- [x] ✅ `numeroDoContratoImportado(numeroProposta, colunaContrato, prefixo)`.
- [x] ✅ `statusObraDaPlanilha(v)` → `ativa | concluida | suspensa | cancelada` (revisão da decisão 2).
- [x] ✅ comentário de `cascataDaExecucao` (linhas 150-154): "regra proposta, a validar com a produção" → "decidido em 2026-10-26 por análise, delegação do Breno (14.1)". Regra sem mudança.

Montagem (`scripts/importacao/montar-registros.mjs`)

- [x] ✅ obras: ler `status` e levar ao registro.
- [x] ✅ array `contratos` (valor = soma dos itens, desconto e condições da principal, status `concluido` se a obra for concluída, senão `ativo`); `aditivo_de` nas outras aprovadas.
- [x] ✅ execuções de item da proposta principal → chave `contrato:<numero>|<item>`; de aditivo ou de proposta não aprovada ficam no item da proposta, com aviso.

- [x] ✅ depois das NFs: aviso por obra com NF `medicao` > 0 e Σ `med_qtd` = 0 ("apontar a medição antes de emitir o relatório"); divergência quando NF de medição > Σ(inst × valor_unit) (decisão 3).
- [x] ✅ FD (linhas 208-211, recomendado): o corte usa só `data_lancamento ?? data_pagamento`, nunca o vencimento; FD só com vencimento depois do corte entra, com aviso (decisão 5).
- [ ] (não feito, opcional) (decisão 1): levar ao cliente as grafias originais diferentes do canônico.

Gravação (`scripts/importacao/gravar.mjs`)

- [x] ✅ obras: gravar `status`.
- [x] ✅ depois dos itens: `garantir('contratos')` pela chave número, pulando obra que já tem contrato não rescindido (aí as aprovadas viram aditivo dele); `created_by` = autor; `data_assinatura` e `doc_url` nulos; observação com a marca `[importacao:…]` e "sem contrato assinado registrado".
- [x] ✅ cópia dos itens da principal para o contrato, chave `contrato|numero`.
- [x] ✅ update de `propostas.contrato_aditivo_id` nos aditivos.
- [x] ✅ `execucao.item_id` apontando para o item do contrato.
- [x] ✅ só se alguma execução vier com med > 0: `execucao_medicoes.data` dessas execuções = data de corte (decisão 3; a tabela não tem policy de escrita, a chave de serviço pode).
- [x] ✅ `desfazerRegistros`: execução dos itens de contrato → itens de contrato → aditivos soltos → contratos, antes de propostas e obras (a FK de obra em contratos é `restrict`).
- [ ] (não feito, opcional) (decisão 1): `clientes.observacao` = "Na planilha também como: ...".

Plano de validação

- [x] ✅ `scripts/importacao/planilha-falsa.mjs`: uma obra com duas propostas aprovadas e uma obra concluída.
- [x] ✅ `scripts/validacao/validar-importacao.mjs`: passos novos (um contrato por obra, aditivo ligado, execução em item de contrato, status da obra, `desfazer` sem contrato sobrando); a fixture de equivalência (linha 41) pode ficar.
- [x] ✅ `scripts/validacao/verificar-sobras.mjs`: achar contratos e itens de contrato da importação.

Sem mudança: `cascataDaExecucao` (regra), `observacaoComMarca`, NF, pagamentos e FD (exceto o corte do FD), `baixar-drive.mjs`.

Documentos

- [x] ✅ `14.2-status-entrega.md`: seção 6 e linha 37 passam de "aguarda a equipe" para "decidido no 14.1".
- [ ] (fica para depois, 14.6) rodar `baixar-drive.mjs` para os 32 arquivos, comprovantes primeiro, quando houver acesso.
- [ ] (registro, 14.4) pedido de mover os arquivos para um Drive da empresa; recomendações da seção 7.

9. Pendências

- **A planilha real:** sem ela, o mapa da seção 6 é suposição, e nenhuma decisão foi exercitada sobre dados reais.
- **Razão social exata** do cliente da Estação Fashion, copiada da NF.
- **Progresso da obra com contrato** (achado da revisão da decisão 2): `progresso_itens_pct` conta os itens de proposta e de contrato no denominador. Fora do 14.1; para a sprint decidir.
- **Validação:** validado em 2026-10-27 (seção 10). Fica pendente só o que depende da planilha real.
- **gc-prod:** fora do escopo por decisão do Breno (26/10); a liberação de prod não será pedida. O go-live fecha como ensaio em gc-dev, e prod é só registro.

10. Validação

Decisões aplicadas e validadas em 2026-10-27. O bloco não muda código por si; as cinco decisões foram aplicadas em 26/10 no importador, na conciliação e no ensaio, e a rodada completa as exercita.

Rodada final de 2026-10-27 (`caffeinate -ims npm run validar`): passou nas sete camadas.

CAMADA	RESULTADO	DESTE BLOCO
estático	limpo (`tsc --noEmit` + `next lint`)	as decisões compiladas no código aplicado
unitário	**432 casos**	`importacao.test.ts` (decisões 1, 3 e 5) e `conciliacao.test.ts`
build	**66 rotas**, iguais à baseline	sem rota nova
runtime	**327/327 rotas** + **348 células** da varredura de URL direta	sem rota nova
dados	**65/65 checagens** (1 sem registro em gc-dev); matriz de RLS: **456 células** de tabela e **108** de bucket	sem query nova
escrita	**529/529 passos**, com o ensaio da importação **17/17** dentro dela	**17/17** do ensaio, que exerce as decisões 1 a 4 (clientes, contrato por obra, cascata, link do Drive); a 5 (corte) vale pelo `--ate`
navegador	**281/281 passos**; screenshots em `/tmp/gc-validacao/shots`	sem tela nova

Continua pendente: o mapa e as decisões nunca rodaram sobre dado real; dependem da "Gestão Contratos.xlsx".